Module 1: Strings and Numbers

JavaScript foundations for web development · Path to Node.js and React

> **Goal of this module:** manipulate text and numbers the way real web apps do: cleaning form input, building URLs, formatting prices, and handling file metadata. Run every example in VS Code with `node filename.js`.

1. Concept recap: strings

```
const title = "  Learn JavaScript  ";

// Basic properties and access
console.log(title.length);        // 20 (spaces count)
console.log(title[2]);            // "L"
console.log(title.at(-3));        // "t" (negative index from the end)

// Cleaning and case
console.log(title.trim());        // "Learn JavaScript"
console.log(title.toUpperCase()); // "  LEARN JAVASCRIPT  "

// Searching
const url = "https://example.com/blog/post-1";
console.log(url.startsWith("https"));  // true
console.log(url.includes("/blog/"));   // true
console.log(url.indexOf("blog"));      // 20 (position, or -1 if absent)

// Extracting
console.log(url.slice(8, 19));    // "example.com"
console.log(url.slice(-6));       // "post-1"

// Splitting and joining
const tags = "js,node,react";
console.log(tags.split(","));                // ["js", "node", "react"]
console.log(["2026", "10", "29"].join("-")); // "2026-10-29"

// Replacing
console.log("hello world".replace("o", "0"));    // "hell0 world" (first only)
console.log("hello world".replaceAll("o", "0")); // "hell0 w0rld"

// Padding (invoice numbers, IDs, clocks)
console.log("42".padStart(6, "0"));  // "000042"
```

> **Key rule:** strings are **immutable**. Methods never change the original; they return a new string. `title.trim()` does nothing unless you store or use the result.

2. Concept recap: numbers

```
// Converting text from forms, URLs, inputs (always strings!)
console.log(Number("19.99"));       // 19.99
console.log(Number("19.99abc"));    // NaN (strict)
console.log(parseFloat("19.99px")); // 19.99 (reads until invalid)
console.log(parseInt("42px", 10));  // 42 (always pass base 10)

// Checking for invalid numbers
console.log(Number.isNaN(Number("abc")));  // true

// The floating-point trap
console.log(0.1 + 0.2);              // 0.30000000000000004
console.log((0.1 + 0.2).toFixed(2)); // "0.30"  <- a STRING

// Rounding
console.log(Math.round(4.5));  // 5
console.log(Math.floor(4.9));  // 4 (down)
console.log(Math.ceil(4.1));   // 5 (up) - useful for page counts
console.log(Math.round(19.987 * 100) / 100); // 19.99 (stays a number)

// Professional currency formatting
const price = 1234.5;
console.log(new Intl.NumberFormat("en-CA",
  { style: "currency", currency: "CAD" }).format(price)); // "$1,234.50"
console.log(new Intl.NumberFormat("fr-CA",
  { style: "currency", currency: "CAD" }).format(price)); // "1 234,50 $"
```

> **Key rule for web dev:** for money, calculate with numbers, round with `Math.round(x * 100) / 100` when needed, and format only at the end for display. Real e-commerce systems often store amounts in **cents** (integers) to avoid floating-point errors.

3. Exercises

Create one file per exercise (`m1-ex1.js`, `m1-ex2.js`, ...). You only need variables and the methods above. No `if` statements yet; conditions come in Module 2.

Exercise 1 — Clean a user's name (profile display)

A user typed `rawName = "  rodrigue TREMBLAY  "` in a sign-up form. Display:

- the cleaned full name: `Rodrigue Tremblay`
- the initials for an avatar: `RT`
- a greeting: `Welcome back, Rodrigue!`

Hint: `trim()`, `split(" ")` (watch out for extra spaces between words; try `split(/\s+/)`), `slice()`, `toUpperCase()`, `toLowerCase()`.

Exercise 2 — Blog post preview

Given a long article text (paste any paragraph of 200+ characters), display:

- a preview of the first 80 characters followed by `…`
- the number of characters and the number of words
- an estimated reading time in minutes, assuming 200 words per minute, rounded **up** (`Reading time: 2 min`)

Hint: `slice()`, `split()`, `.length`, `Math.ceil()`.

Exercise 3 — URL slug generator

Blogs and e-commerce sites turn titles into URL-friendly slugs. Convert:

```
"  Top 10 JavaScript Tips & Tricks for 2026!  "
  -> "top-10-javascript-tips-tricks-for-2026"
```

Then build the full URL:
`https://myblog.com/posts/top-10-javascript-tips-tricks-for-2026`

Hint: chain methods: `trim()` → `toLowerCase()` → *remove special characters with* `replace(/[^a-z0-9\s-]/g, "")` → *replace spaces with hyphens using* `replace(/\s+/g, "-")`.

Exercise 4 — Shopping cart line with Quebec taxes

Form inputs always arrive as strings: `priceInput = "19.99"`, `quantityInput = "3"`. Calculate and display, formatted in both `en-CA` and `fr-CA`:

- subtotal
- GST (5%) and QST (9.975%), each rounded to the cent
- total

Expected subtotal: `$59.97` / `59,97 $`.

Hint: convert with `Number()` *first, round each tax with* `Math.round(x * 100) / 100`*, and format with* `Intl.NumberFormat`*. Bonus: redo the calculation in cents (integers) and compare.*

Exercise 5 — File upload summary

A user uploads a file named `"Rapport_Final.v2.PDF"` with a size of `5347840` bytes, and the server assigns it ID `42`. Display:

```
File ID:    FILE-000042
Name:       Rapport_Final.v2
Extension:  pdf
Size:       5.10 MB
Storage:    uploads/rapport_final.v2-000042.pdf
```

Hint: the extension is after the **last** *dot:* `lastIndexOf(".")`*. 1 MB = 1024 × 1024 bytes.* `padStart()` *for the ID.*

4. Sources to prepare and go further

Read before the exercises

- MDN: Handling text — strings in JavaScript — beginner-friendly introduction
  https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Scripting/Strings
- javascript.info: Strings — clear explanations with small tasks at the end
  https://javascript.info/string
- javascript.info: Numbers — covers rounding and the floating-point problem very well
  https://javascript.info/number

Reference while coding (bookmark these)

- MDN: String reference — every string method with examples
  https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/String
- MDN: Number reference — conversion and checks
  https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Number
- MDN: Math reference — rounding and math functions
  https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Math
- MDN: Intl.NumberFormat — currency and locale formatting
  https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Intl/NumberFormat

To deepen

- Eloquent JavaScript, chapter 1: Values, Types, and Operators — free online book, excellent for solid foundations
  https://eloquentjavascript.net/01_values.html
- MDN: Regular expressions guide — only the basics are needed for exercise 3
  https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Regular_expressions
- regex101.com — test and understand regular expressions visually
  https://regex101.com

> **Next step:** send your code for review once you finish one or more exercises. Then Module 2: conditions and logic (form validation, user roles, default values).